CYBER SECURITY PROJECT · FINAL REVIEW

Explainable, Visibility-Aware Runtime Threat Detection for Containerized Linux Workloads

Falco · Tracee · Tetragon

Canonical schema + evidence graph

Visibility-aware verdicts

Vansh Bhatiya — 23BCE0631 · Mahendra Sajjan Chaudhary — 23BCE0701

02 · THE PROBLEM

Alerts Without Context

1

Fragmented telemetry

Falco, Tracee and Tetragon each emit a different event format.

2

Telemetry ≠ detection

A raw event is visibility; it is not a verdict that something is malicious.

3

Visibility gaps

“No event observed” is NOT proof the workload was safe.

▸ Confusing telemetry, detection and collection-quality causes two failures: missing events read as safety (false confidence) and noisy scores read as attacks (false alarms). TraceWeaver keeps the three separate.

03 · WHAT WE BUILT

One Pipeline, Three Signals, Evidence-Linked Verdicts

EXISTING SENSORS (NOT OURS)

- Falco — rule-based alerting engine
- Tracee — events + behavioural signatures
- Tetragon — eBPF process/file tracing

BUILT IN TRACEWEAVER (VERIFIED)

- Canonical schema + normalization — one shape across 3 sensors
- Behaviour graph + evidence correlation — reused by the detector
- Three signals → weighted risk → verdict — A · S · P, visibility-gated
- Evidence provenance + interactive replay — every claim → file:line:sha256

▸ Partial / honest scope: Tetragon live container capture is broken (COLLECTION_FAILED); the app runs over recorded evidence (Historical Replay), not a live kernel feed.

04 · SYSTEM ARCHITECTURE

From Runtime Events to Evidence-Linked Output



External tools (not ours)

Python (engine + pipeline)

Next.js / TypeScript (UI + API)

Historical replay enters at 'Raw evidence'

`pipeline.py :: run() → run.py :: run_all() → engine.py :: run_engine()`

Raw Sensor Event → Canonical Event

- Per-sensor adapters — falco_adapter / tracee_adapter / tetragon_adapter map raw JSON → canonical FIELDS
- Schema validation + quarantine — validate_event() rejects bad records to quarantine.jsonl (currently 0)
- Deterministic event_id — sha256(sensor|trial|file|line|native|timestamp)[:20] — reproducible
- raw_ref provenance — {file, line, sha256} links every event to its source line + file hash
- Explicit missing values — sentinels like <NOT_APPLICABLE> — never guess an unknown fact

Real example — Tracee sudoers (raw_ref → source line 8)

```
raw: results/raw/tracee_events/A6_tracee_t1.jsonl:8 → eventName:"sudoers_modification"
canonical: {
"event_id":"ev_bb1d8efbf63e8779844d", "canonical_type":"security_alert",
"sensor":"tracee", "sensor_native_event":"sudoers_modification", "process_name":"sh",
"file_path":"<NOT_APPLICABLE>", "timestamp":"2026-10-16T06:51:06.932777+00:00",
"raw_ref":{"file":"...A6_tracee_t1.jsonl","line":8,"sha256":"3c002c…"} }
```

06 · BEHAVIOUR GRAPH

Correlation Model Built From Evidence Only

NODES

- process — key proc:<cid12>:<pid>; unknown identity stays explicit
- file — key file:<path> on file ops
- network — endpoint on connect
- security_event — alert-only process node

EDGES (EVIDENCE-BACKED)

- parent_of — needs concrete pid + parent_pid
- accessed — process → file on file ops
- connected — process → endpoint
- in_container — derived from a real container_id

▸ Missing pid/parent_pid → nodes stay disconnected; the engine does NOT invent lineage. Dangling edges to an unobserved parent are dropped. A graph edge is the available evidence, not proof of causality; temporal proximity alone is never an edge.

Backend Truth, Frontend Rendering

- One source of truth — graph_adapter.py reuses the engine's BehaviorGraph — no second detector
- Payload — ordered events + nodes + edges + nested engine result → graph API
- Custom SVG renderer — BehaviorGraph.tsx — NO external graph library
- Deterministic layout — graphLayout.ts :: computeLayout (index-seeded force layout, no RNG)
- Progressive replay — Investigation.tsx — Play/Pause/Step/Reset/speed/scrubber; node reveals at its first event, edge at its supporting event
- Synchronized — timeline + inspector track the current event; selecting a finding shows its evidence

▸ Labelled “Historical Replay” everywhere — playback of recorded evidence in real timestamp order, not live monitoring.

Three Independent Signals → One Risk Score

A
Anomaly
share of suspicious actions vs a fitted baseline

S
Suspicious sequence
max severity of temporal behaviour patterns

P
Policy evidence
1 iff a real qualifying rule/signature fired

build graph → score anomaly → A → detect patterns + score sequence → S → score policy → P → combine → risk → assess visibility → compute verdict

▸ Deterministic rules + frequency statistics — NOT a black-box ML model. Chosen for auditability and reproducibility on a small dataset.

score_anomaly() → A ∈ [0, 1]

$$A = \min(1.0 , \text{anomalous_count} / \max(1, \text{total_events}) \times 3.0) \text{ (round 4 dp)}$$

- Anomalous if — file_path in sensitive_paths (/etc/shadow, /etc/sudoers, /etc/ssh/ssh_host_rsa_key, /etc/gshadow)
- …or — a process_exec arg runs from a suspicious dir (/tmp/, /dev/shm/, /var/tmp/)
- Novel process name alone — does NOT raise A — recorded as context only (avoids flagging benign echo/ls)
- max(1, …) guard — prevents divide-by-zero; ×3 scales a small fraction, capped at 1.0
- No events → A = 0.0 — absence is never treated as anomalous (nor as safety)

detect_patterns() + score_sequence() → S = max severity

PATTERN	SEVERITY	TRIGGER
sensitive_file_access	0.80	file op on a sensitive path
sudoers_modification	0.75	file op on /etc/sudoers*
exec_from_suspicious_dir	0.70	exec arg in a temp/world-writable dir
nested_shell_execution	0.45 / 0.30	≥2 shells; 0.45 only with a parent_of chain
discovery_sequence	0.40	≥3 distinct discovery tools

▸ S = max(severities), not a sum — avoids inflating risk when several weak patterns co-occur. Patterns require specific canonical types + fields; a missing parent-child relationship lowers nested_shell to 0.30 (never invented).

score_policy() → P ∈ {0, 1}

P = 1.0 iff a qualifying security_alert exists; otherwise P = 0.0

- Qualifying — a Falco built-in rule or a Tracee behavioural signature (canonical_type = security_alert)
- Excluded — the lab's custom notice rule “EBPF Lab Container Process Execution” — telemetry, not detection
- Binary — P is 0 or 1; the qualifying events + their raw_ref are attached to the result
- Why — ordinary telemetry is observation, not a confirmed detection — DETECTED requires P > 0

Weighted, Bounded, Configurable

$$R = \text{round}(100 \times (0.2 \cdot A + 0.3 \cdot S + 0.5 \cdot P) , 2) \rightarrow R \in [0, 100]$$

WEIGHTS (ENGINE_CONFIG.JSON, SUM = 1.0)

- wA = 0.2 — anomaly — weakest / most environment-dependent
- wS = 0.3 — sequence
- wP = 0.5 — policy — most evidence-grounded
- Bands — LOW < 40 ≤ SUSPICIOUS < 70 ≤ HIGH

WORKED — A6_TRACEE_T1 (REAL TRIAL)

- A = 0.0, S = 0.75, P = 1.0 — from the engine on this stream
- R = 100·(0.2·0 + 0.3·0.75 + 0.5·1) — = 100 × 0.725
- R = 72.5 — band = HIGH
- Verdict — DETECTED (P > 0)

Risk ≠ Verdict: Collection Quality Comes First

1 Execution failed?	→ EXECUTION_FAILED
2 Collection failed?	→ COLLECTION_FAILED
3 No events?	→ INCONCLUSIVE
4 Qualifying policy (P > 0)?	→ DETECTED
5 S > 0 or risk ≥ 40?	→ TELEMETRY_ONLY
6 Full visibility, nothing fired?	→ NOT_DETECTED
otherwise	→ INCONCLUSIVE

▸ A high risk score can NEVER promote a collection/execution failure to a detection. NOT_DETECTED means nothing qualifying fired under full visibility — it is not proof of safety.

From Click to Evidence-Linked Outcome

Select stream
(Main · Scenario Explorer)
→
Preview graph
POST /graph
→
Run Analysis
POST /graph (live run)
→
Engine executes
cli graph → run_engine()
→
Result returned
nodes·edges·verdict
→
Outcome renders
verdict·risk·A/S/P
→
Graph replay + inspector
evidence provenance

- /api/traceweaver/run — runs the engine, returns the EngineResult
- /api/traceweaver/graph — runs the engine + graph adapter, returns the full investigation payload
- Security — allowlist from events_index.json + path-traversal guard + no shell + 60s timeout

Combined Scoring Recovers Recall Rules-Only Misses

52	46	6	0
eligible trials	positive cases	held-out benign	false positives

METHOD	TP	FP	TN	FN	PRECISION	RECALL	F1
Rules-only	20	0	6	26	1.000	0.435	0.606
Anomaly-only	33	0	6	13	1.000	0.717	0.835
Combined (TraceWeaver)	43	0	6	3	1.000	0.935	0.966

▸ Caveat: only SIX benign negatives — too few to establish a real-world false-positive rate. Historical-replay figures; not live latency/throughput. Not independently validated.

A Defensible Prototype Contribution

LIMITATIONS

- Small benign set — 6 negatives — no reliable FP rate
- Historical replay — not validated fully live
- Tetragon capture — live container capture broken
- Correlation — weaker when identifiers/clocks are missing

CONTRIBUTION

- Integration — 3 sensors → one canonical schema
- Visibility-aware verdicts — risk never overrides failure
- Evidence contract — finding → event_id → file:line:sha256
- Explainable replay — auditable, deterministic, no ML box

▸ TraceWeaver connects heterogeneous runtime telemetry to explainable, visibility-aware security findings — a prototype research contribution, not a production IDS.

TECHNICAL APPENDIX

Filesystem map · code navigation · worked calculations · graph example · evaluation detail · commands · Q&A

APPENDIX A · FILESYSTEM MAP

Where Everything Lives

PATH	ROLE
scripts/traceweaver/engine.py	detection engine — graph, A/S/P, risk, verdict
scripts/traceweaver/engine_config.json	weights (.2/.3/.5) + thresholds (40/70) + anomaly lists
scripts/traceweaver/schema.py	canonical schema, sentinels, build/validate event, event_id
scripts/traceweaver/adapters.py	falco/tracee/tetragon → canonical
scripts/traceweaver/pipeline.py	normalization driver (raw → canonical + index)
scripts/traceweaver/run.py	runner + anomaly-baseline fit (no leakage)
scripts/traceweaver/graph_adapter.py	read-only behaviour-graph payload
scripts/traceweaver/evaluate.py · cli.py	metrics · CLI (run/graph/validate)
dashboard/components/traceweaver/*.tsx	OutcomeView · Investigation · BehaviorGraph
results/{raw,normalized,engine}/	evidence · canonical events · engine output
tests/ (test_engine · test_traceweaver · test_graph_adapter)	51 automated tests

APPENDIX B · CODE NAVIGATION

Open-the-Code Cheat Sheet

FACULTY QUESTION	FILE	SYMBOL / COMMAND
Show your algorithm	engine.py	grep -n 'def score_' scripts/traceweaver/engine.py
Where is A / S / P	engine.py	score_anomaly / detect_patterns+score_sequence / score_policy
Risk formula + weights	engine.py · engine_config.json	grep -n 'risk = round' …/engine.py
Final verdict / visibility	engine.py	compute_verdict / assess_visibility
Normalize / schema / IDs	pipeline.py · schema.py	run() / build_event() / _stable_event_id
Behaviour graph / edges	engine.py	class BehaviorGraph (_build)
Graph payload / SVG / replay	graph_adapter.py · *.tsx	build_graph_payload / BehaviorGraph.tsx / Investigation.tsx
API that runs analysis	route.ts	app/api/traceweaver/graph/route.ts
Evaluation / tests	evaluate.py · tests/	predict()+metrics() / pytest tests/

Line numbers drift — use grep -n to locate symbols.

APPENDIX C · WORKED CALCULATION

One Real Trial, End to End — A6_tracee_t1

STEP	VALUE
Events in stream	9
Anomalous events	0 → A = 0.0
Triggered pattern	sudoers_modification (0.75) → S = 0.75
Qualifying policy	Tracee signature present → P = 1.0
Risk	100·(0.2·0 + 0.3·0.75 + 0.5·1.0) = 72.5
Risk band	HIGH (≥ 70)
Visibility	FULL
Verdict	DETECTED (P > 0)
Provenance	signature event @ results/raw/tracee_events/A6_tracee_t1.jsonl:8

▸ All values from a single recorded trial — reproduce with: python3 -m scripts.traceweaver.cli run --stream results/raw/tracee_events/A6_tracee_t1.jsonl --no-write

APPENDIX D · GRAPH EXAMPLE

A1_falco_t1 — Sparse but Honest

- Input — 3 Falco security_alert events (sh, cat, cat); pid/parent_pid NOT supplied
- Nodes — 3 security_event nodes + 1 container node (real container_id)
- Edges — 3 in_container edges (derived from container_id); engine edges = 0
- No parent_of / accessed — identity fields missing → NOT fabricated
- UI — shows the connected nodes + a “why correlation is limited: pid, parent_pid missing” panel

> ▸ Contrast A6_tracee_t1: process+file+container nodes with parent_of/accessed edges and an emphasised finding node. E04 (Tetragon) → COLLECTION_FAILED, 0 nodes: a distinct failure state, never an empty 'clean' graph.

APPENDIX E · EVALUATION DETAIL

Confusion Matrices, Labels, Reproducibility

- **Labels** — A1/A5/A6 = positive · A7 = negative · A2/A3 = ambiguous (excluded) · E-series excluded
- **Predict** — rules: P>0 · anomaly: A≥0.2 · combined: P>0 or strong pattern (≥0.6) or A≥0.2
- **No leakage** — baseline fit only on A2/A3+E-series (excluded from metrics); A7 held out as negatives
- **Metrics** — precision=TP/(TP+FP), recall=TP/(TP+FN), F1=harmonic mean; zero denom → N/A
- **Provenance** — evaluation.json carries git rev d02ce97, engine 2.0.0, config_hash 2e474f93

▸ Reproduce: python3 -m scripts.traceweaver.evaluate → results/engine/evaluation.{json,md}. Six benign negatives: preliminary, not a validated FP rate.

APPENDIX F · DEMO COMMANDS

Verified Terminal Commands

Run / test / inspect (from ~/ebpf-evasion-lab)

```
cd ~/ebpf-evasion-lab && source .toolchain/activate.sh
cd dashboard && npm run build && PORT=3000 npm run start # open http://localhost:3000
./scripts/traceweaver_demo.sh # 8-step engine demo
python3 -m pytest tests/ -q # 51 passed
python3 -m scripts.traceweaver.cli run --stream results/raw/tracee_events/A6_tracee_t1.jsonl --no-write
python3 -m scripts.traceweaver.cli graph --stream results/raw/tracee_events/A6_tracee_t1.jsonl | less
sed -n '142,275p' scripts/traceweaver/engine.py # the A/S/P algorithm
cat -n scripts/traceweaver/engine_config.json # weights + thresholds
cat results/engine/evaluation.md # metrics
```

Do not assume the server is already running — start it with the command above.

APPENDIX G · HARD QUESTIONS (1/2)

Defensible Answers

Q · Is this machine learning?

No — deterministic rules + frequency statistics. Chosen for auditability and determinism on a tiny dataset; ML would overfit and obscure.

Q · Why three sensors?

Different coverage — Falco catches some rules, Tracee has signatures (e.g. sudoers), Tetragon traces. One schema lets us compare them fairly.

Q · Why max severity, not sum?

A single strong pattern shouldn't be diluted; stacking weak patterns shouldn't manufacture a high score.

Q · How do you avoid fabricated edges?

Edges need concrete identity fields; missing pid/parent_pid → disconnected nodes + an explanation, never an invented link.

Q · What is actually live?

The engine runs live on demand; the sensor events are recorded (Historical Replay). No fresh capture in the demo.

APPENDIX G · HARD QUESTIONS (2/2)

Defensible Answers

Q · How do you prove a finding is real?

Every finding lists event_ids; every event has raw_ref {file,line,sha256}; the CLI validator proves no finding cites a missing event and no DETECTED lacks P>0.

Q · What happens when the sensor fails?

Visibility = COLLECTION_FAILED → verdict COLLECTION_FAILED regardless of score; the UI shows a distinct failure state, not a clean negative.

Q · How are false positives measured?

On the held-out benign control (A7): 0 FP across all methods — but only 6 negatives, so it's preliminary, not a real-world rate.

Q · What's the novelty?

The integration: unified schema + visibility-aware verdict + end-to-end evidence contract, proven by validators and 51 tests.

Q · Production readiness?

Not yet — needs live streaming ingestion, a large benign corpus, hardened identity resolution, and throughput/latency/overhead measurement.